

Bài 2: Mô hình hóa miền (Domain modeling)

Nội dung chính

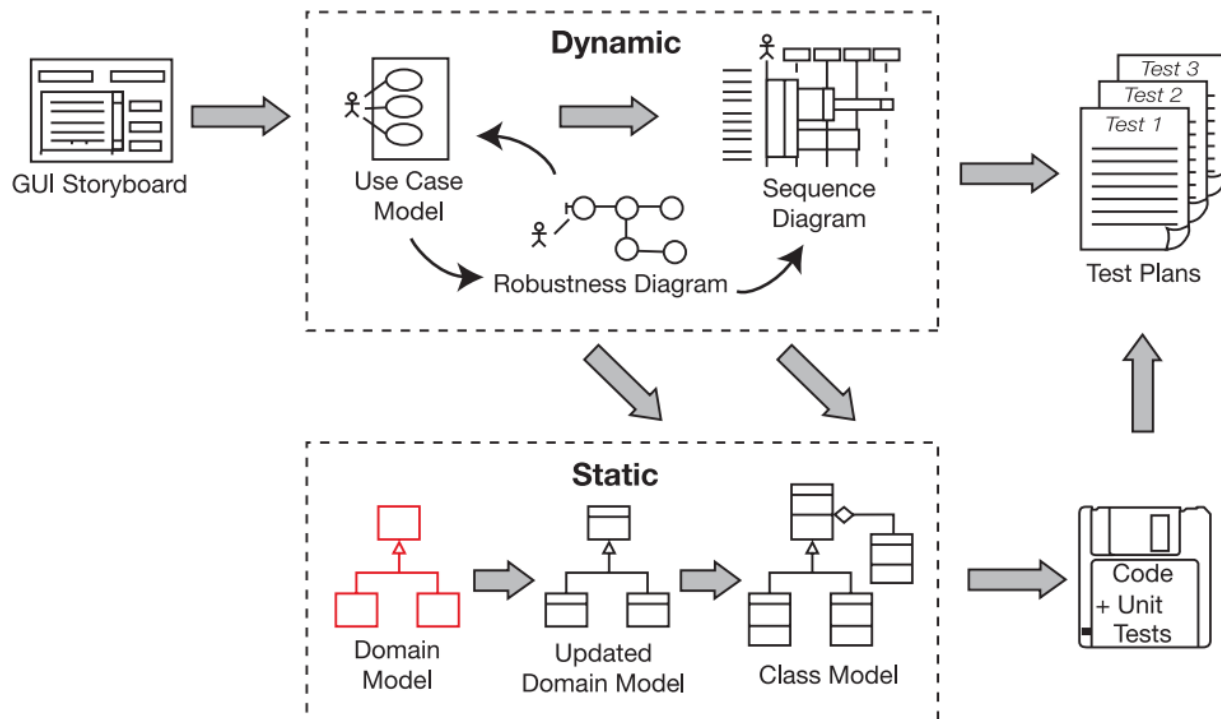
Bài này tập trung vào việc giới thiệu và hướng dẫn phương pháp mô hình hóa miền (Domain Modeling) — một bước nền tảng trong quy trình ICONIX, giúp kết nối yêu cầu nghiệp vụ với thiết kế phần mềm một cách rõ ràng, thống nhất và có hệ thống. Domain model được xem như “bản đồ khái niệm” phản ánh thế giới thực mà phần mềm cần mô phỏng, đóng vai trò là ngôn ngữ chung giữa khách hàng, nhà phân tích, lập trình viên và tester. Bài học trình bày bối cảnh ra đời và vai trò của domain model trong việc lấp đầy khoảng trống giao tiếp đầu dự án, phân biệt domain model với database model, cũng như lý do tại sao domain modeling phải được thực hiện trước hoặc song song với use case. Tiếp theo, bài học trình bày các nguyên tắc cốt lõi và quy trình 7 bước để xây dựng một domain model hiệu quả — từ trích xuất danh từ trong yêu cầu, chuẩn hóa thuật ngữ, xác định quan hệ has-a / is-a / knows-about, đến tinh chỉnh mô hình qua các vòng lặp. Phần cuối của bài học áp dụng các nguyên tắc này vào case study Internet Bookstore, minh họa cách phát triển domain model qua ba vòng tinh chỉnh, từ bản sơ khai đến mô hình hoàn chỉnh có cấu trúc rõ ràng, hỗ trợ trực tiếp cho giai đoạn use case và thiết kế chi tiết.

Mục tiêu cần đạt

- Hiểu được khái niệm, vai trò và lợi ích của domain modeling trong phát triển phần mềm hướng đối tượng.
- Phân biệt rõ domain model với database model và nhận biết rủi ro khi nhầm lẫn hai khái niệm này.
- Nắm vững các loại quan hệ chính trong domain model (has-a, is-a, knows-about) và cách áp dụng chúng.
- Thực hiện được quy trình 7 bước xây dựng domain model, từ phân tích yêu cầu đến tinh chỉnh mô hình.
- Vận dụng phương pháp domain modeling vào bài toán thực tế thông qua case study Internet Bookstore.
- Nhận thức được vai trò then chốt của domain model như “xương sống tĩnh” cho toàn bộ quy trình thiết kế và lập trình sau này.
- Phát hiện và loại bỏ các khái niệm mơ hồ, trùng lặp hoặc không thuộc miền bài toán ngay từ giai đoạn đầu dự án.

2.1. Bối cảnh và vai trò

Khi bắt đầu phát triển một hệ thống phần mềm, bước đầu tiên thường khiến các nhóm dự án bối rối nhất không phải là chọn ngôn ngữ lập trình hay framework nào, mà là làm sao để tất cả mọi người thực sự hiểu đúng “chúng ta đang xây cái gì”. Ở giai đoạn đầu, khách hàng có ý tưởng, nhóm phân tích có cách diễn giải riêng, lập trình viên có hình dung khác, còn tester lại hiểu theo một cách nữa. Nếu không có một công cụ chung để thống nhất ngôn ngữ và khái niệm, dự án dễ dàng rơi vào hỗn loạn ngay từ vòng gửi xe.



Đây chính là lúc Domain Modeling (mô hình hóa miền) trở thành “cầu nối ngôn ngữ” quan trọng. Các bạn có thể hình dung Domain Model như một bản đồ khái niệm của thế giới thực mà phần mềm sẽ phản ánh. Nó không phải là thiết kế kỹ thuật, càng không phải là sơ đồ cơ sở dữ liệu, mà là một cách nắm bắt các khái niệm cốt lõi của bài toán, tổ chức chúng lại thành một mô hình có thể giao tiếp được với mọi bên liên quan.

2.1.1. Khoảng trống giao tiếp trong phát triển phần mềm

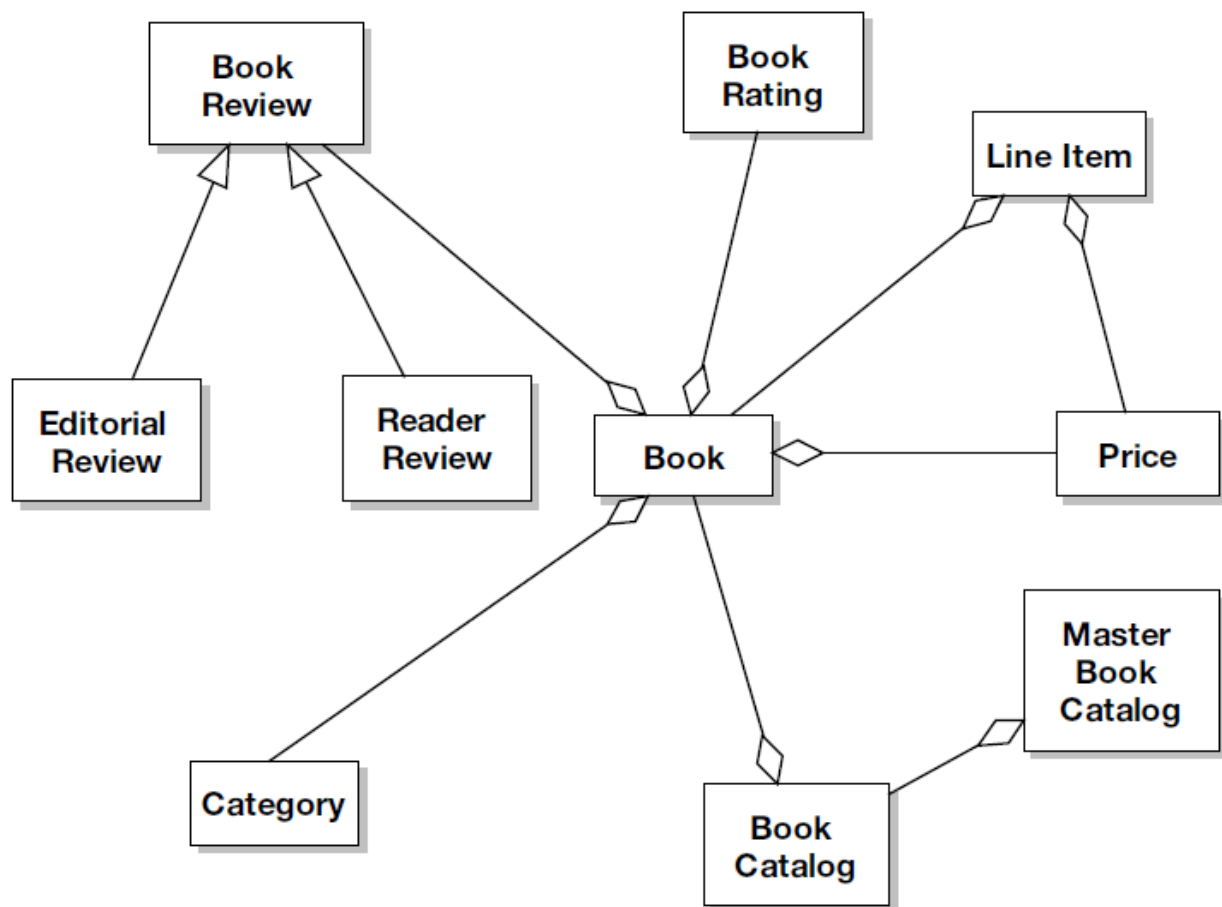
Hãy tưởng tượng các bạn tham gia vào một dự án phát triển hệ thống “Internet Bookstore” – một cửa hàng sách trực tuyến. Khách hàng mô tả yêu cầu rất đơn giản:

“Tôi muốn một website bán sách trực tuyến. Khách hàng có thể tìm sách, thêm vào giỏ hàng và thanh toán. Tôi cũng muốn quản lý tồn kho và thông tin sách.”

Nghe thì có vẻ rõ ràng, nhưng khi nhóm bắt đầu làm việc, mỗi người lại hiểu một kiểu khác:

- Lập trình viên backend bắt đầu thiết kế bảng Book, Cart, Order theo cách riêng.
- Nhóm frontend dựng giao diện mà không chắc cần những trường nào.
- Tester chưa hiểu rõ quy trình thêm sách vào giỏ hàng có các bước trung gian gì.
- Quản lý dự án thì chỉ muốn thấy giao diện demo nhanh nhất có thể.

Kết quả là mọi người nói cùng một ngôn ngữ tự nhiên, nhưng lại không chia sẻ cùng một “mô hình tinh thần” về hệ thống. Đây là nguyên nhân gốc rễ của rất nhiều lỗi hiểu nhầm, thiết kế chông chéo, và chi phí sửa lỗi tăng cao ở giai đoạn sau. Domain Model chính là công cụ được đưa ra để giải quyết triệt để khoảng trống giao tiếp này.



Mô hình miền (domain model) là một công cụ sống và mang tính hợp tác. Nó được tinh chỉnh và cập nhật liên tục trong suốt quá trình của dự án, để luôn phản ánh sự hiểu biết hiện tại về không gian vấn đề.

2.1.2. Domain Model như “từ điển chung” của dự án

Các bạn hãy nghĩ về Domain Model như **một cuốn từ điển mà cả nhóm cùng biên soạn và cùng sử dụng**. Trong cuốn từ điển này, mỗi khái niệm của bài toán được xác định rõ:

- Tên gọi (ví dụ: “Book”, “Author”, “ShoppingCart”)
- Mối quan hệ với các khái niệm khác (một cuốn sách có nhiều tác giả, một tác giả có thể viết nhiều sách, một giỏ hàng chứa nhiều cuốn sách...)

Khi mọi người đều dùng chung cuốn từ điển này, việc thảo luận trở nên chính xác và nhanh hơn nhiều. Lập trình viên và tester không còn tranh cãi về “sách” là gì hay “đơn hàng” được định nghĩa thế nào, vì tất cả đã được mô hình hóa rõ ràng trong domain model. Điều quan trọng là domain model không phải do một người thiết kế ra trong cô lập. Nó được xây dựng thông qua đối thoại giữa nhóm phân tích, khách hàng, lập trình viên và những người hiểu bài toán thực tế. Quá trình này giúp phát hiện ra những chỗ hiểu sai, những khái niệm còn mơ hồ, và cả những yêu cầu tiềm ẩn mà khách hàng chưa diễn đạt được.

2.1.3. Domain Model khác gì Database Model?

Một sai lầm rất phổ biến của sinh viên và cả nhiều nhóm startup là nhảy thẳng vào vẽ bảng cơ sở dữ liệu và gọi đó là domain model. Thực tế, domain model và data model là hai thứ hoàn toàn khác nhau, mặc dù có thể có những điểm tương đồng.

- Domain Model là mô hình khái niệm. Nó tập trung vào “*ý nghĩa trong thế giới thực*” của các khái niệm. Ví dụ: “Book” là một thực thể có quan hệ với “Author” và “Publisher”.
- Data Model là mô hình vật lý của dữ liệu trong hệ thống. Nó tập trung vào “*cách lưu trữ và truy xuất dữ liệu*”. Ví dụ: bảng BOOK có các cột ISBN, TITLE, AUTHOR_ID.

Domain model có thể tồn tại mà không cần biết phần mềm sẽ dùng MySQL hay MongoDB. Nó là bước trước khi quyết định kỹ thuật triển khai, và vì thế nó đóng vai trò định hướng cho toàn bộ thiết kế phía sau.

2.1.4. Vì sao Domain Modeling được đặt lên trước Use Case trong ICONIX?

Nếu các bạn còn nhớ Chương 1, ICONIX Process có một điểm rất khác với nhiều phương pháp truyền thống:

Domain Modeling được thực hiện trước hoặc song song với việc viết use case.

Tại sao vậy? Vì domain model giúp nhóm xây dựng ngôn ngữ chung và nhận diện các khái niệm cốt lõi ngay từ đầu. Khi viết use case sau đó, các bạn sẽ có sẵn một “bộ từ vựng chuẩn” để mô tả hành vi hệ thống. Điều này giúp tránh tình trạng mỗi use case lại dùng thuật ngữ khác nhau cho cùng một khái niệm — một lỗi cực kỳ phổ biến trong các dự án thực tế.

Ví dụ: nếu domain model đã xác định có thực thể “ShoppingCart”, thì tất cả use case sau đó sẽ thống nhất dùng “ShoppingCart” thay vì người này viết “Cart”, người kia viết “Basket”, người khác lại nói “Danh sách sách đã chọn”. Nhờ đó, thiết kế sau này không bị phân mảnh.

Hãy dành 2 giờ xây dựng domain model trước khi viết use case để tránh thuật ngữ mơ hồ gây sai lệch về sau.

2.1.5. Domain Model là nền móng cho toàn bộ thiết kế

ICONIX xem domain model là “xương sống tĩnh” (static backbone) của toàn bộ hệ thống.

- Trong giai đoạn yêu cầu, domain model giúp hiểu bài toán và thống nhất ngôn ngữ.
- Trong giai đoạn robustness analysis, domain model cung cấp danh sách entity để gắn vào sơ đồ tính vững chắc.
- Trong giai đoạn sequence diagram, domain model phát triển thành class diagram với các phương thức và tương tác cụ thể.
- Trong giai đoạn coding, domain model định hướng cấu trúc lớp và cơ sở dữ liệu.

Nếu domain model ban đầu sai hoặc hời hợt, toàn bộ quy trình sau đó sẽ như một tòa nhà xây trên nền móng yếu: càng lên cao càng dễ sụp.

Tác giả ICONIX kể rằng trong nhiều dự án thật, khi nhóm bỏ qua bước domain modeling, hậu quả thường không đến ngay lập tức mà xuất hiện ở giai đoạn thiết kế chi tiết hoặc coding: Lớp bị trùng lặp hoặc đặt tên không nhất quán. Dữ liệu quan trọng bị bỏ sót. Các controller phải “chữa cháy” cho những lỗi khái niệm từ đầu. Việc tích hợp module trở nên khó khăn vì không có mô hình chung để tham chiếu. Ngược lại, các dự án có domain model tốt thường có tài liệu ngắn gọn nhưng thiết kế rõ ràng, quá trình phát triển trơn tru và dễ bảo trì hơn hẳn.

Trước khi viết bất kỳ use case hay dòng code nào, các bạn nên ngồi lại và hỏi: “Chúng ta đã thực sự thống nhất về các khái niệm của bài toán chưa?” Nếu câu trả lời là chưa, thì Domain Modeling chính là nơi để bắt đầu.

2.2. Nguyên tắc và quy trình xây dựng domain model

Ở phần trước, các bạn đã hiểu rằng domain model đóng vai trò như “*từ điển chung*” của dự án, là cây cầu nối ngôn ngữ giữa khách hàng, phân tích viên, lập trình viên và tester. Vậy câu hỏi tiếp theo là: *làm thế nào để xây dựng được một domain model đúng đắn, hữu ích, không bị sa đà vào chi tiết kỹ thuật, cũng không quá mơ hồ để không dùng được?* Phần này sẽ giúp các bạn nắm được các nguyên tắc cốt lõi và quy trình thực tế để tạo nên một domain model hiệu quả.

2.2.1. Bắt đầu từ ngôn ngữ tự nhiên của yêu cầu

Một trong những điểm khác biệt quan trọng của Domain Modeling so với thiết kế kỹ thuật là điểm xuất phát. Khi các bạn thiết kế hệ thống, thường sẽ bắt đầu từ những khái niệm kỹ thuật như lớp (class), phương thức (method), database table, framework... Nhưng khi xây dựng domain model, điểm khởi đầu là ngôn ngữ tự nhiên mà khách hàng sử dụng để mô tả bài toán.

Hãy quay lại ví dụ Internet Bookstore. Khách hàng có thể mô tả:

“Chúng tôi bán sách qua Internet. Khách hàng có thể tìm kiếm sách theo tiêu đề, tác giả hoặc ISBN. Họ có thể thêm sách vào giỏ hàng và thanh toán bằng thẻ tín dụng. Chúng tôi cũng muốn theo dõi tồn kho và thông tin nhà xuất bản.”

Từ đoạn mô tả này, nếu các bạn gạch chân những danh từ quan trọng, các bạn sẽ thu được danh sách các khái niệm miền bài toán ban đầu:

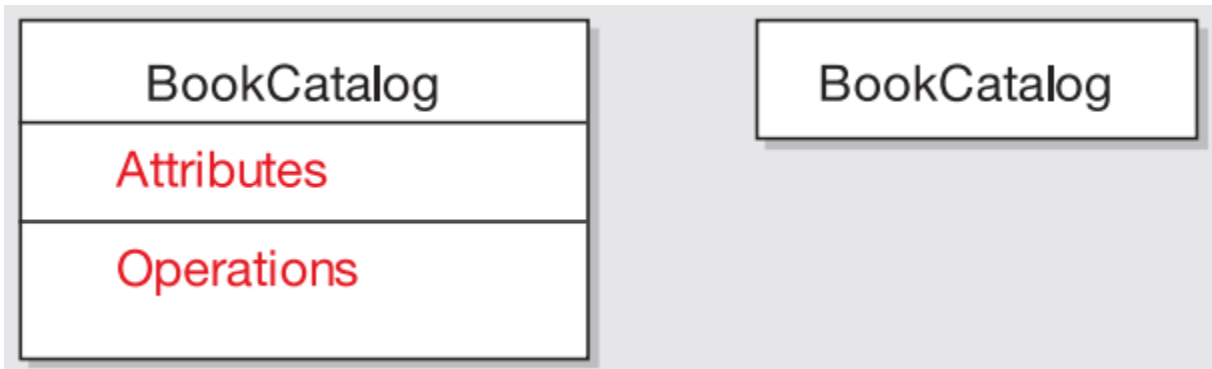
- Book (Sách)
- Author (Tác giả)
- ISBN
- Customer (Khách hàng)
- Shopping Cart (Giỏ hàng)
- Credit Card (Thẻ tín dụng)
- Inventory (Tồn kho)
- Publisher (Nhà xuất bản)

Đây chính là “nguyên liệu thô” để bắt đầu xây dựng domain model. Trong các dự án thật, danh sách này thường được lấy từ tài liệu yêu cầu, email, cuộc họp, hoặc đơn giản là phỏng vấn khách hàng.

Nguyên tắc 1: *Hãy để yêu cầu dẫn dắt domain model, không để kỹ thuật dẫn dắt.*

2.2.2. Giữ mô hình ở mức khái niệm, chưa phải thiết kế kỹ thuật

Một lỗi phổ biến mà sinh viên hay mắc là đi vào chi tiết quá sớm. Ngay khi thấy “Book”, họ bắt đầu nghĩ đến các trường book_id, title, price... và mối quan hệ foreign key trong database. Điều này là quá sớm cho giai đoạn domain modeling.



Domain model chỉ nên thể hiện các khái niệm và mối quan hệ ở mức khái niệm (conceptual), chứ chưa phải thiết kế (design). Ví dụ, giữa Book và Author, các bạn chỉ cần thể hiện mối quan hệ “một tác giả có thể viết nhiều sách, một sách có thể có nhiều tác giả”. Không cần thiết phải xác định tên bảng trung gian hay kiểu dữ liệu.

Nguyên tắc 2: *Domain Model là conceptual model — mô hình khái niệm, không phải chi tiết kỹ thuật.*

2.2.3. Ưu tiên mô hình hóa “danh từ”, tránh lẫn “động từ”

Trong ngôn ngữ tự nhiên, danh từ thường đại diện cho **các thực thể (entity)** hoặc khái niệm miền; còn động từ thường đại diện cho hành vi hoặc chức năng, sẽ xuất hiện ở use case hoặc sequence diagram sau này.

Một số sinh viên thường nhầm và đưa cả động từ vào domain model. Ví dụ, họ có thể thêm “Checkout” (Thanh toán) hay “Search” (Tìm kiếm) như một lớp trong domain model. Đây là lỗi. “Checkout” là một hành vi, nó sẽ được mô hình hóa ở phần robustness hoặc sequence diagram.

Nguyên tắc 3: *Bắt đầu từ danh từ (concept), chưa đưa động từ (behavior) vào domain model.*

2.2.4. Loại bỏ trùng lặp và chuẩn hóa thuật ngữ

Khi các bạn trích xuất danh sách khái niệm từ yêu cầu, thường sẽ có nhiều từ gần nghĩa hoặc đồng nghĩa. Ví dụ:

- “Shopping Cart” và “Basket” có thể cùng nói về giỏ hàng.
- “Customer” và “User” đôi khi dùng lẫn lộn.

- “Bookstore” có thể vừa chỉ website, vừa chỉ toàn bộ doanh nghiệp.

Nếu không chuẩn hóa, domain model sẽ trở nên lộn xộn và gây nhầm lẫn về sau. Do đó, nhóm cần ngồi lại và thống nhất một từ duy nhất cho mỗi khái niệm, chọn tên rõ ràng, tránh viết tắt, và ghi chú nếu cần.

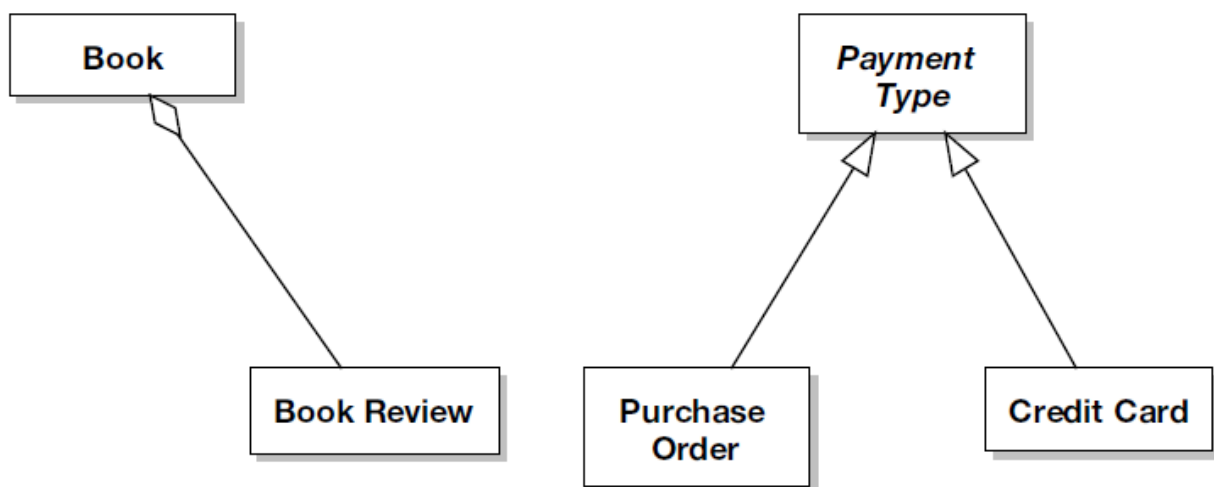
Nguyên tắc 4: *Chuẩn hóa tên gọi để tránh trùng lặp và nhầm lẫn.*

Đây là một kỹ thuật cực kỳ quan trọng và được sử dụng rất nhiều dạng của lĩnh vực xử lý ngôn ngữ tự nhiên. Ở học phần này các bạn sẽ hợp nhất các khái niệm thủ công – chạy bằng côm – sang học phần khác chúng ta sẽ dùng thư viện, công cụ để giải quyết vấn đề này

2.2.5. Xác định mối quan hệ cốt lõi

Sau khi có danh sách lớp miền đã được chuẩn hóa, bước tiếp theo là xác định các mối quan hệ quan trọng giữa chúng. Trong domain modeling, có hai loại quan hệ rất thường gặp (95%):

1. Generalization (IS-A) – quan hệ tổng quát hóa: Ví dụ: Book có thể chia thành EBook và PrintedBook. Đây là quan hệ “là một loại của”.
2. Aggregation / Composition (HAS-A) – quan hệ kết hợp: Ví dụ: ShoppingCart chứa nhiều Book. Đây là quan hệ “có chứa”.
3. Knows about: liên kết thông thường giữa hai lớp.



Việc thể hiện rõ các mối quan hệ giúp domain model không chỉ là một danh sách rời rạc các khái niệm, mà trở thành một mạng lưới có cấu trúc phản ánh thế giới thực.

Nguyên tắc 5: *Xác định và thể hiện rõ quan hệ IS-A và HAS-A giữa các khái niệm.*

2.2.6. Tránh đưa yếu tố giao diện và kỹ thuật vào mô hình

Một lỗi khác rất phổ biến là sinh viên đưa cả các lớp như LoginForm, SearchButton, PaymentAPI vào domain model. Những lớp này không thuộc về miền bài toán, mà thuộc về tầng giao diện hoặc kỹ thuật triển khai. Domain model chỉ nên tập trung vào các khái niệm nghiệp vụ cốt lõi.

Nguyên tắc 6: *Không đưa GUI hoặc thành phần kỹ thuật vào domain model.*

2.2.7. Xây dựng mô hình lặp và tinh chỉnh dần

Domain modeling không phải là một bước làm một lần rồi bỏ. Trong thực tế, mô hình sẽ được tinh chỉnh dần qua các vòng lặp:

- Lần đầu: phác thảo sơ khai từ yêu cầu.
- Lần hai: chỉnh sửa khi viết use case.
- Lần ba: cập nhật khi phân tích robustness.
- Lần bốn: bổ sung khi thiết kế sequence diagram.

ICONIX khuyến khích cách tiếp cận lặp này để đảm bảo domain model phản ánh đúng hiểu biết hiện tại của nhóm về bài toán, đồng thời không bị “đóng băng” quá sớm.

Nguyên tắc 7: *Xây dựng domain model theo vòng lặp — tinh chỉnh khi hiểu biết tăng lên.*

2.2.8. Quy trình xây dựng domain model thực tế

Từ các nguyên tắc trên, ta có thể rút ra một **quy trình thực tế** để xây dựng domain model như sau:

1. Thu thập yêu cầu sơ khởi – đọc tài liệu, phỏng vấn khách hàng, ghi chú danh từ quan trọng.
2. Trích xuất danh sách lớp miền – từ yêu cầu, chọn ra các danh từ cốt lõi.
3. Chuẩn hóa và loại bỏ trùng lặp – thống nhất tên gọi.
4. Xác định quan hệ IS-A và HAS-A – vẽ sơ đồ khái niệm cơ bản.
5. Kiểm tra loại bỏ yếu tố GUI/kỹ thuật – giữ mô hình ở mức khái niệm.
6. Rà soát với nhóm và khách hàng – để phát hiện chỗ hiểu nhầm.
7. Tinh chỉnh lặp – cập nhật khi có thêm thông tin.

Quy trình này có thể áp dụng cho bất kỳ miền bài toán nào, từ hệ thống bán hàng, quản lý thư viện cho đến hệ thống ngân hàng hoặc giáo dục.

Một domain model tốt không phải là một sơ đồ cực kỳ chi tiết, mà là một sơ đồ:

- Gọn gàng, dễ hiểu, không bị lẫn yếu tố kỹ thuật.
- Bao phủ được các khái niệm quan trọng của bài toán.
- Có quan hệ rõ ràng, tên gọi nhất quán.
- Có thể dùng để viết use case và làm cơ sở cho thiết kế.

Nếu các bạn có thể đưa domain model cho một thành viên mới trong nhóm và họ hiểu ngay hệ thống nói về cái gì, thì đó là dấu hiệu tốt.

2.3. Case study: Internet bookstore

Yêu cầu:

1. **Hiệu sách** sẽ ban đầu được triển khai trên web, nhưng kiến trúc của nó phải linh hoạt để có thể phát triển các giao diện khác (như Swing, applet, dịch vụ web, v.v.).
2. Hiệu sách phải có khả năng bán **sách**, với các **đơn hàng** được chấp nhận qua **Internet**.
3. Người dùng phải có thể thêm sách vào **giỏ hàng trực tuyến** trước khi **thanh toán**.
 - a. Tương tự, người dùng phải có thể xóa các **mục** khỏi giỏ hàng.
4. Người dùng phải có thể duy trì **danh sách mong muốn** với các sách họ muốn mua sau này.
5. Người dùng phải có thể hủy đơn hàng trước khi chúng được giao.
6. Người dùng phải có thể thanh toán bằng **thẻ tín dụng** hoặc **đơn đặt hàng**.
7. Hệ thống phải cho phép người dùng trả lại sách.
8. Hiệu sách phải có khả năng tích hợp vào các trang web của **đối tác** thông qua các **danh mục nhỏ**, được lấy từ một **danh mục tổng** lưu trữ trong **cơ sở dữ liệu** trung tâm.
 - a. Các danh mục nhỏ phải được định nghĩa bằng XML vì chúng sẽ được chuyển giao giữa hệ thống này và các hệ thống bên ngoài (sẽ được định nghĩa sau).
 - b. **Hệ thống giao hàng** sẽ được thực hiện thông qua Amazon Web Services.
9. Người dùng phải có thể tạo **tài khoản khách hàng** để hệ thống ghi nhớ thông tin của họ (tên, địa chỉ, chi tiết thẻ tín dụng) khi đăng nhập.
 - a. Hệ thống sẽ duy trì **danh sách tài khoản** trong cơ sở dữ liệu trung tâm.

- b. Khi người dùng đăng nhập, **mật khẩu** của họ phải luôn được đối chiếu với danh sách mật khẩu trong **danh sách tài khoản chính**.
10. Người dùng phải có thể tìm kiếm sách bằng nhiều **phương thức** khác nhau như **tiêu đề, tác giả, từ khóa** hoặc **thể loại**, và sau đó xem **chi tiết sách**.
11. Người dùng phải có thể đăng các đánh giá về những cuốn sách yêu thích; phần **bình luận** sẽ xuất hiện trên màn hình chi tiết sách. Đánh giá phải bao gồm **khách hàng xếp hạng** từ 1 đến 5, thường được hiển thị cùng với tiêu đề sách trong **danh sách sách**.
- a. **Đánh giá sách** phải được kiểm duyệt — tức là, được kiểm tra và phê duyệt bởi nhân viên trước khi xuất hiện trên website.
- b. Những đánh giá dài sẽ bị cắt ngắn trên màn hình chi tiết sách; **người dùng** có thể nhấp để xem toàn bộ đánh giá trên một trang riêng.
12. Nhân viên cũng phải có thể đăng các **đánh giá biên tập** về sách. Những đánh giá này cũng sẽ xuất hiện trên màn hình chi tiết sách.
13. Hiệu sách phải cho phép các **người bán** bên thứ ba (ví dụ: các hiệu sách bán đồ cũ) thêm **danh mục sách** của riêng họ. Những sách này sẽ được thêm vào **danh mục tổng** để xuất hiện trong kết quả tìm kiếm.
14. Hiệu sách phải có khả năng mở rộng, với các yêu cầu cụ thể sau:
- a. Hiệu sách phải có khả năng duy trì **tài khoản người dùng** cho 100.000 khách hàng trong 6 tháng đầu, và sau đó là thêm 1.000.000 khách hàng.
- b. Hiệu sách phải có khả năng phục vụ tới 1.000 người dùng đồng thời (10.000 sau 6 tháng).
- c. Hiệu sách phải có khả năng xử lý tới 100 yêu cầu tìm kiếm mỗi phút (1.000/phút sau 6 tháng).
- d. Hiệu sách phải có khả năng xử lý tới 100 lượt mua hàng mỗi giờ (1.000/giờ sau 6 tháng).

Dưới đây là danh sách các danh từ và cụm danh từ được trích xuất từ yêu cầu của hệ thống hiệu sách trực tuyến, sau khi đã chuyển số nhiều thành số ít và sắp xếp theo thứ tự bảng chữ cái:

1. Đối tác (**Associate Partner**)

2. Tác giả (**Author**)

3. Sách (**Book**)
4. Danh mục sách (**Book Catalog**)
5. Chi tiết sách (**Book Details**)
6. Danh sách sách (**Book List**)
7. Đánh giá sách (**Book Review**)
8. Hiệu sách (**Bookstore**)
9. Thể loại (**Category**)
10. Thanh toán (**Checkout**)
11. Thẻ tín dụng (**Credit Card**)
12. Khách hàng (**Customer**)
13. Tài khoản khách hàng (**Customer Account**)
14. Xếp hạng khách hàng (**Customer Rating**)
15. Cơ sở dữ liệu (**Database**)
16. Đánh giá biên tập (**Editorial Review**)
17. Mạng Internet (**Internet**)
18. Mục hàng (**Item**)
19. Từ khóa (**Keyword**)
20. Danh sách tài khoản (**List of Accounts**)
21. Danh sách tài khoản chính (**Master Account List**)
22. Danh mục sách tổng (**Master Book Catalog**)
23. Danh mục chính (**Master Catalog**)
24. Danh mục nhỏ (**Mini-Catalog**)
25. Đơn hàng (**Order**)
26. Mật khẩu (**Password**)
27. Đơn đặt hàng (**Purchase Order**)
28. Bình luận đánh giá (**Review Comment**)
29. Phương thức tìm kiếm (**Search Method**)
30. Kết quả tìm kiếm (**Search Results**)
31. Người bán (**Seller**)
32. Hệ thống thực hiện giao hàng (**Shipping Fulfillment System**)
33. Giỏ hàng (**Shopping Cart**)
34. Tiêu đề (**Title**)
35. Tài khoản người dùng (**User Account**)
36. Danh sách mong muốn (**Wish List**)

Danh sách này chứa khá nhiều sự trùng lặp; các thuật ngữ tương tự đang được sử dụng cho những khái niệm cơ bản giống nhau. Tuy nhiên, đây chính là lợi ích chính của phương pháp mô hình hóa miền: ta có cơ hội nhận diện và loại bỏ những thuật ngữ trùng lặp này ngay từ đầu dự án.

Một số mục trong danh sách đơn giản là không cần thiết vì chúng nằm ngoài phạm vi của mô hình miền, hoặc chúng là hành động đang được ngay trang thành danh từ. Hãy cùng đi qua danh sách này và tinh chỉnh nó một chút:

- Ta có thể nghĩ rằng các thuật ngữ "Customer" (Khách hàng) và "Customer Account" (Tài khoản khách hàng) là trùng lặp, nhưng thực tế chúng đại diện cho hai khái niệm khác nhau: "Customer Account" là một thực thể được lưu trữ trong cơ sở dữ liệu, trong khi "Customer" là một tác nhân (actor).
- "Customer" và "Seller" (Người bán) là các tác nhân, và do đó nên được đặt trên các sơ đồ use case.
- Các thuật ngữ ~~"User Account"~~ và "Customer Account" là trùng lặp. Việc chọn giữ lại thuật ngữ nào là khá tùy ý, vì vậy chúng ta sẽ chọn "Customer Account."
- Các thuật ngữ ~~"List of Accounts"~~ (Danh sách tài khoản) và "Master Account List" (Danh sách tài khoản tổng) là trùng lặp, vì vậy chúng ta nên loại bỏ một trong số chúng. Vì chúng ta cũng có "Master Book Catalog" (Danh mục sách tổng), điều hợp lý là giữ lại "Master Account List."
- Các thuật ngữ "Book Review" (Đánh giá sách) và ~~"Review Comment"~~ (Bình luận đánh giá) là trùng lặp, vì vậy chúng ta sẽ giữ "Book Review."
- Chúng ta có một vài thuật ngữ khác nhau để chỉ danh mục hoặc danh sách sách: "Book Catalog" (Danh mục sách), "Book List" (Danh sách sách), "Mini-Catalog" (Danh mục nhỏ), và "Master Catalog" (Danh mục tổng). Danh mục và danh sách có thể là những khái niệm khác nhau. Thực tế, có vẻ như các yêu cầu đang cố gắng truyền tải điều gì đó, có thể chỉ ám chỉ trong văn bản. Khi có nghi ngờ, hãy hỏi khách hàng. Hỏi cho đến khi ta nhận được câu trả lời rõ ràng và không mơ hồ.
 - ~~"Book Catalog"~~ và "Master Catalog" thực sự là trùng lặp ở đây, vì vậy chúng ta sẽ giữ lại "Master Catalog," vì nó có sự tương phản tốt với "Mini-Catalog." "Book List," trong khi đó, có thể là một thuật ngữ bao quát cho các loại danh sách khác nhau; chúng ta sẽ giữ lại nó ở đây và xem cách nó phù hợp khi vẽ sơ đồ mô hình miền.
- Có một sự trùng lặp khác ở khu vực này: "Master Catalog" và "Master Book Catalog." Chúng ta sẽ xóa ~~"Master Catalog"~~ vì "Master Book Catalog" mô tả chi tiết hơn.
- Từ ~~"Internet"~~ quá chung chung và không thêm giá trị gì ở đây.
- Từ "Password" (Mật khẩu) quá nhỏ để trở thành một đối tượng và sẽ được hiển thị như một thành phần giao diện người dùng, vì vậy chúng ta nên loại bỏ nó khỏi mô hình miền. Nếu bắt đầu đưa tất cả các thành phần giao diện người

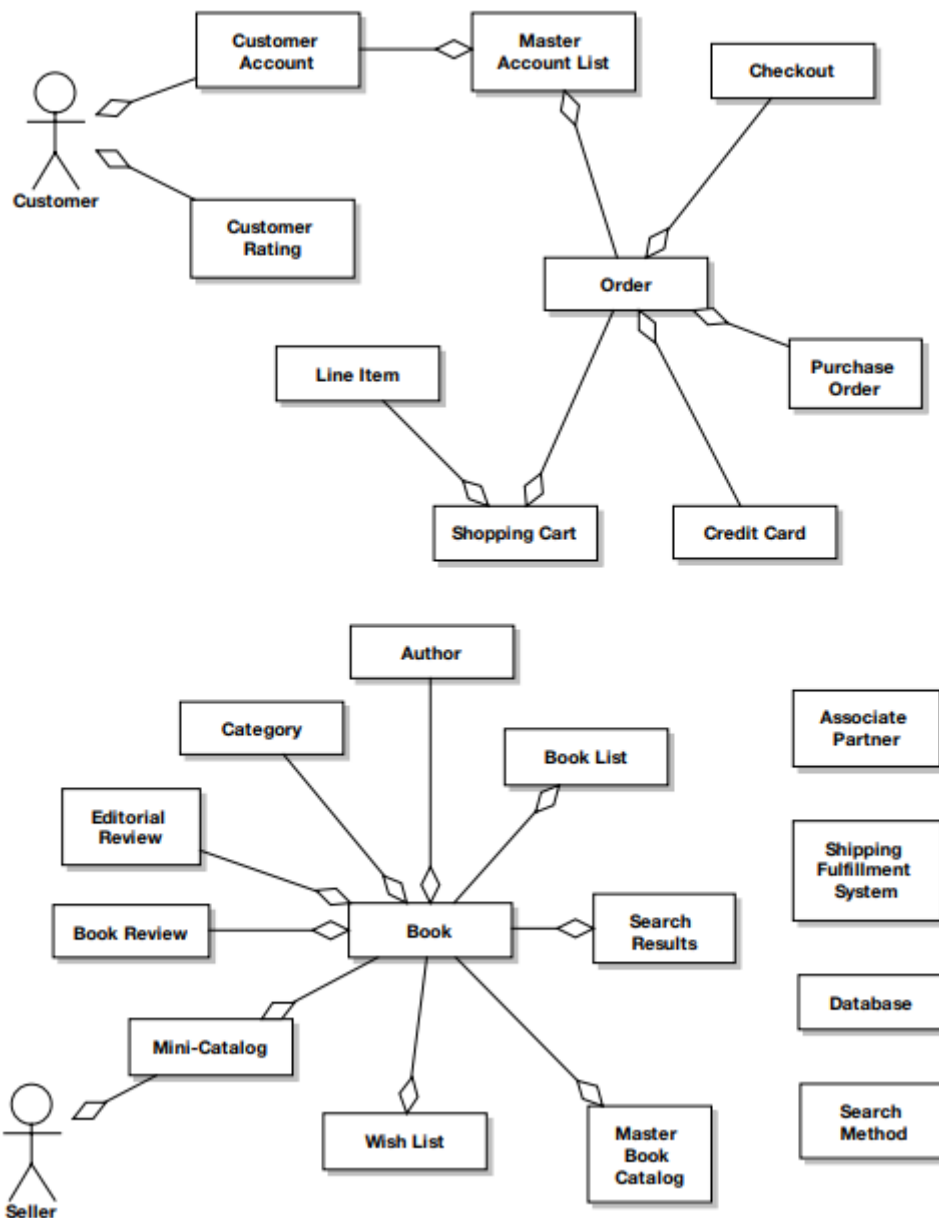
dùng vào mô hình miền, chúng ta sẽ mở ra một rắc rối lớn và có thể sẽ phải xử lý cả đêm với rất nhiều vấn đề phát sinh.

- Tương tự, chúng ta cũng nên loại bỏ "**Title**" (Tiêu đề) và "**Keyword**" (Từ khóa).
- Một sự trùng lặp nữa là "**Book**" và "**Book Details**" (Chi tiết sách). Chúng ta sẽ chỉ giữ lại "**Book**," vì nó súc tích hơn "**Book Details**" mà không làm mất ý nghĩa.
- Từ "**Item**" (Mục hàng) quá mơ hồ, nhưng nó đại diện cho một khái niệm hợp lệ: một mục hàng đã được thêm vào giỏ hàng của người dùng. Vì vậy, chúng ta sẽ đổi tên nó thành "**Line Item**" và giữ lại trong danh sách.
- Từ "**Bookstore**" (Hiệu sách) quá rộng và khó có thể được đề cập trực tiếp, vì vậy chúng ta có thể loại bỏ nó.

Dưới đây là danh sách các lớp miền tiềm năng đã được tinh chỉnh:

1. **Associate Partner** (Đối tác liên kết)
2. **Author** (Tác giả)
3. **Book** (Sách)
4. **Book List** (Danh sách sách)
5. **Book Review** (Đánh giá sách)
6. **Category** (Thể loại)
7. **Checkout** (Thanh toán)
8. **Credit Card** (Thẻ tín dụng)
9. **Customer Account** (Tài khoản khách hàng)
10. **Customer Rating** (Xếp hạng khách hàng)
11. **Database** (Cơ sở dữ liệu)
12. **Editorial Review** (Đánh giá biên tập)
13. **Line Item** (Mục hàng)
14. **Master Account List** (Danh sách tài khoản tổng)
15. **Master Book Catalog** (Danh mục sách tổng)
16. **Mini-Catalog** (Danh mục nhỏ)
17. **Order** (Đơn hàng)
18. **Purchase Order** (Đơn đặt hàng)
19. **Search Method** (Phương thức tìm kiếm)
20. **Search Results** (Kết quả tìm kiếm)
21. **Shipping Fulfillment System** (Hệ thống thực hiện giao hàng)
22. **Shopping Cart** (Giỏ hàng)
23. **Wish List** (Danh sách mong muốn)

Công việc tiếp theo là phác họa đồng này lên sơ đồ.



2.3.1. Xây dựng sơ đồ mô hình miền ban đầu (Building the First Domain Model)

Bước kế tiếp sau khi tinh chỉnh danh sách các lớp miền là xây dựng sơ đồ domain model ban đầu. Ở giai đoạn này, mục tiêu không phải là tạo ra một sơ đồ hoàn hảo, mà là xác định các mối quan hệ sơ khai giữa những khái niệm cốt lõi, thường thông qua các quan hệ aggregation (has-a), generalization (is-a) và association (knows-about).

a. Thiết lập các quan hệ “has-a” (aggregation)

Một kỹ thuật hữu ích là **đọc sơ đồ thành tiếng**, chèn cụm “has-a” vào giữa hai lớp, để kiểm tra tính hợp lý.

Ví dụ, ta có thể nói:

- “Shopping Cart has Line Items” → nghe hợp lý → quan hệ chứa đựng
- “Book Catalog has Books” → hợp lý → mỗi danh mục chứa nhiều sách
- “Order has Checkout” → không hợp lý → Checkout không phải lớp miền, mà là hành vi

Dựa trên danh sách lớp đã tinh chỉnh, ta có thể xác định một số quan hệ has-a chính ban đầu như sau:

- ShoppingCart has-a LineItem
- BookCatalog has-a Book
- MasterBookCatalog has-a BookCatalog
- ShoppingCart has-a BookList (thông qua các mục đã chọn)
- Order has-a LineItem và has-a CustomerAccount
- WishList has-a Book (danh sách mong muốn chứa nhiều sách)

Các quan hệ này tạo thành xương sống cấu trúc của sơ đồ.

b. Nhóm tạm thời các lớp “mồ côi” (unconnected classes)

Một số lớp hiện tại chưa có quan hệ rõ ràng, ví dụ như:

- AssociatePartner (đối tác liên kết),
- ShippingFulfillmentSystem (hệ thống giao hàng),
- Database,
- SearchMethod.

Các lớp này tạm thời được nhóm ở phía bên phải sơ đồ, để chờ giai đoạn phân tích robust (robustness analysis) sau đó. Lúc này, nhóm sẽ quyết định:

- Liên kết chúng với các lớp khác, hoặc
- Đổi vai trò thành actor (tác nhân bên ngoài), hoặc
- Loại bỏ nếu chúng không thuộc miền bài toán cốt lõi.

2.3.2. Tinh chỉnh mô hình miền qua brainstorm nhóm (Second Attempt through Brainstorming)

Trong các dự án thật, bản domain model đầu tiên thường chỉ là điểm khởi đầu. Khi nhóm thực hiện các buổi brainstorming, nhiều lớp miền mới sẽ xuất hiện từ hiểu biết miền của thành viên, không nhất thiết phải có trong yêu cầu gốc.

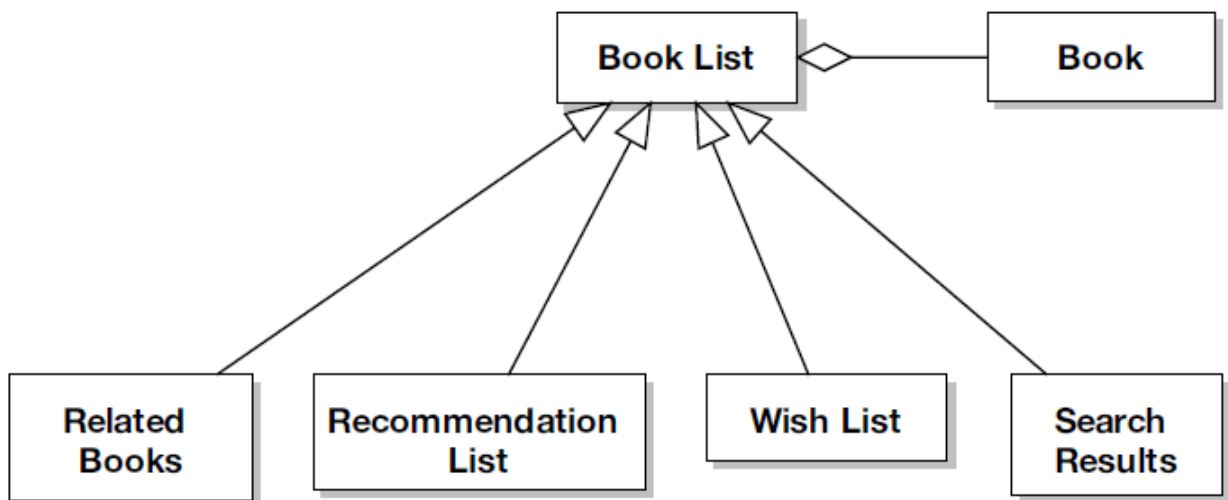
Ví dụ trong case Internet Bookstore:

- OrderHistory (Lịch sử đơn hàng) được thêm để hỗ trợ chức năng xem lịch sử mua sắm.
- OrderDispatch (Giao đơn hàng) được thêm để mô tả quá trình fulfillment.

Đồng thời, một số lớp được loại bỏ hoặc điều chỉnh:

- Checkout bị loại bỏ vì nó là hành vi, không phải lớp miền.
- Author bị loại bỏ vì chỉ là thuộc tính nhỏ của Book.
- Quan hệ giữa Book và MasterBookCatalog được tinh chỉnh bằng cách thêm lớp trung gian BookCatalog. → Một Book thuộc về một BookCatalog, và BookCatalog thuộc về MasterBookCatalog.

Đây chính là bản chất lặp & tinh chỉnh liên tục của ICONIX



2.3.3. Tổng quát hoá (Generalization) để đơn giản hóa cấu trúc

Ở lần tinh chỉnh thứ ba, nhóm áp dụng quan hệ tổng quát hoá (is-a) để gom các lớp cùng loại, giảm rối sơ đồ.

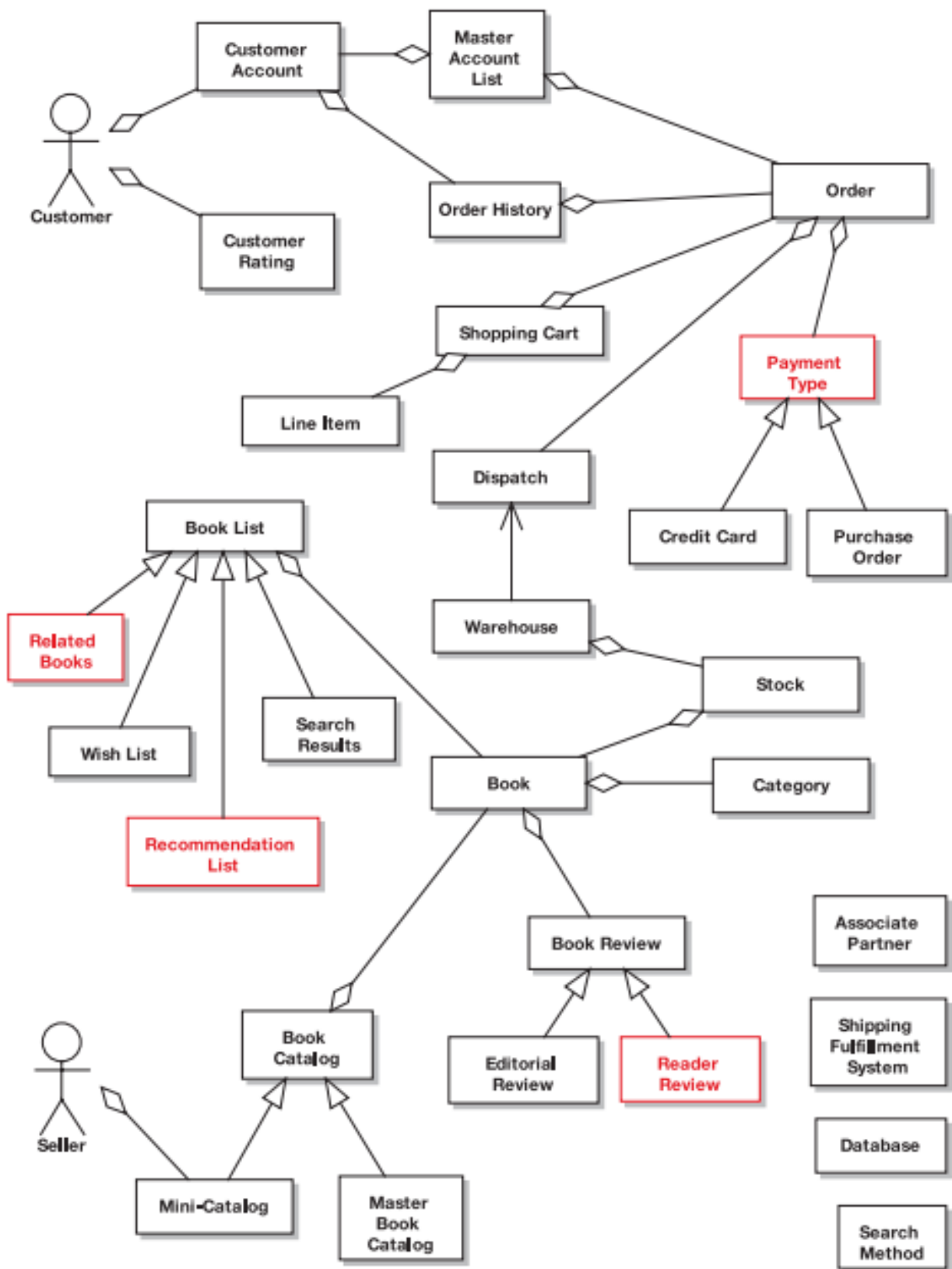
Ví dụ:

- MiniCatalog và MasterBookCatalog → đều là các loại BookCatalog.
- WishList, RecommendationList, RelatedBooks, SearchResults → đều kế thừa từ BookList.
- EditorialReview và ReaderReview kế thừa từ BookReview.
- CreditCard và PurchaseOrder kế thừa từ PaymentType.

Kết quả là mô hình miền trở nên mạch lạc, phân cấp rõ ràng, và dễ mở rộng về sau.

2.3.4. Sơ đồ mô hình miền hoàn chỉnh (Domain Model Diagram)

Hình dưới đây minh họa sơ đồ domain model của hệ thống Internet Bookstore sau ba vòng tình chỉnh — bao gồm các quan hệ has-a, is-a, và knows-about.



2.4. Ví dụ tương tự

Chúng ta hãy xem một ví dụ tương tự sau với

- Đầu vào: yêu cầu hệ thống (giống chúng ta vừa làm xong)
- Đầu ra: Sơ đồ lớp (cũng là cái chúng ta muốn trong khoá học này):

Problem Statement

"Develop a graphic editor that can draw different geometric shapes such as line, circle and triangle. User can select, move or rotate a shape. To do so, editor provides user with a menu listing different commands. Individual shapes can be grouped together and can behave as a single shape."

Step 1). Identify Classes

Extract nouns in the problem statement.

Develop a graphic **editor** that can draw different geometric **shapes** such as **line**, **circle** and **triangle**. **User** can select, move or rotate a **shape**. To do so, **editor** provides **user** with a **menu** listing different **commands**. Individual **shapes** can be grouped together and can behave as a single **shape**.

Eliminate irrelevant classes.

- Editor - Very broad scope
- User – Out of system boundary
- commands – Broad scope

Add more classes by analyzing requirements

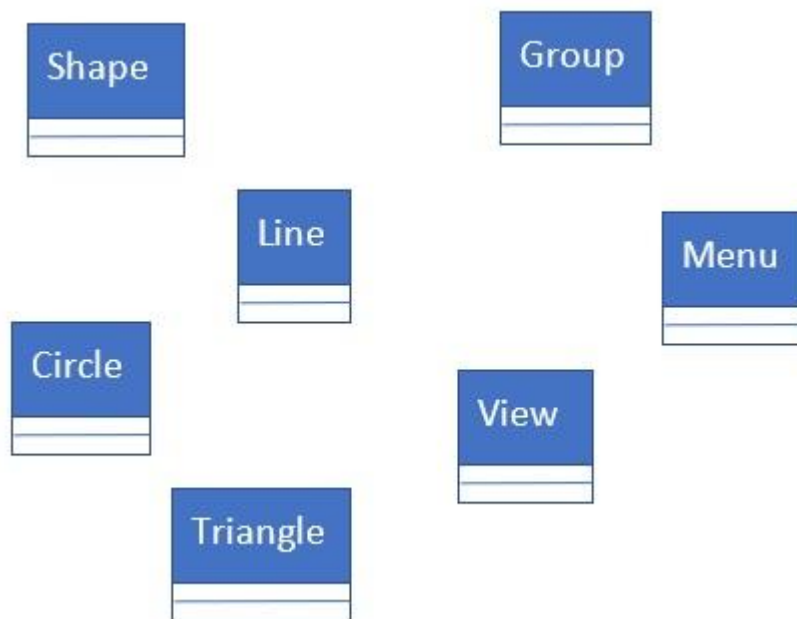
- Group - required to behave as a shape
 - Individual shapes can be grouped together and can behave as a single shape
- View – editor must have a display area

Following classes have been identified:

- Shape
- Line
- Circle
- Triangle
- Menu
- **Group**

- **View**

Basic Object Model - Graphic Editor



Step 2). Identify Associations

Extract verbs connecting objects.

"Individual shapes can be grouped together"

- Group consists of lines, circles, triangles
- Group can also consists of other groups

(Composition)

Verify access paths.

View contains shapes

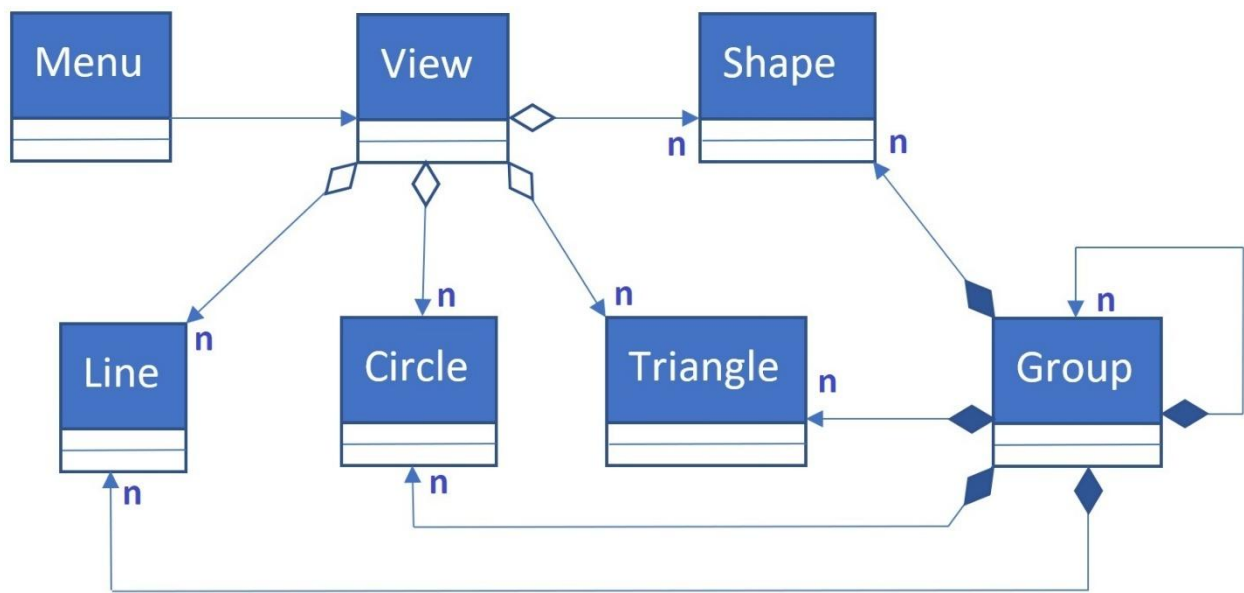
- View contains lines
- View contains circles
- View contains triangles
- View contains groups

(Aggregation)

Menu sends message to View

(Simple One-Way Association)

Basic Object Model - Graphic Editor



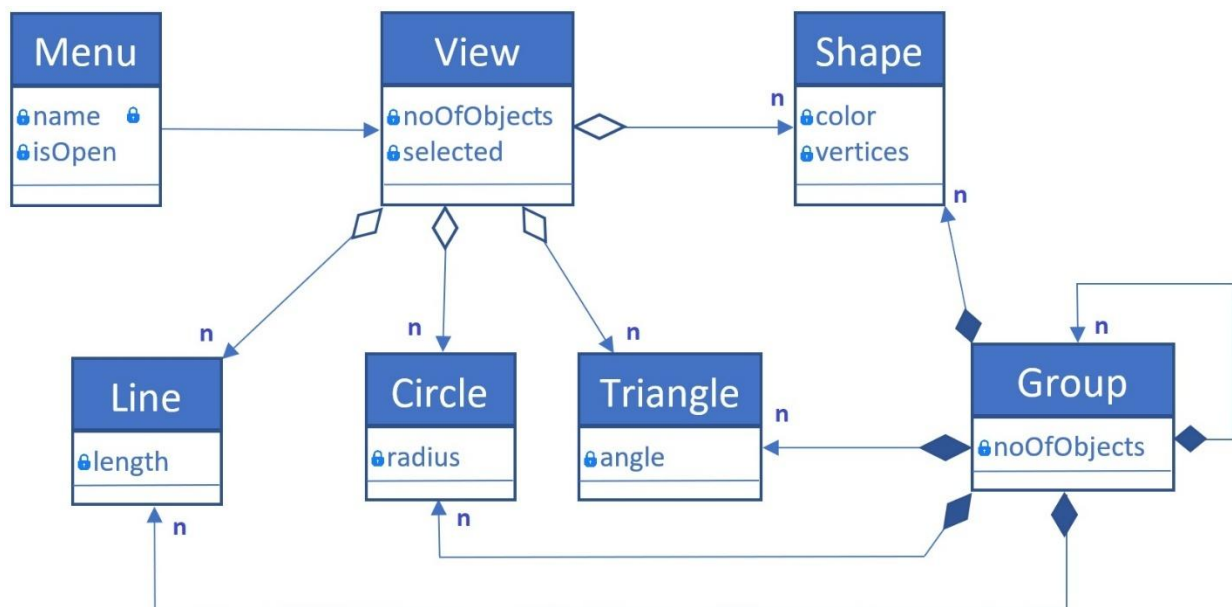
Step 3). Identify Attributes

Extract properties of the object from the domain knowledge.

- **Shape**
 - Color
 - Vertices
- **Line**
 - Color
 - Vertices
 - Length
- **Circle**
 - Color
 - Vertices
 - Radius
- **Triangle**
 - Color
 - Vertices
 - Angle

- **Menu**
 - Name
 - isOpen
- **Group**
 - noOfObjects
- **View**
 - noOfObjects
 - selected

Basic Object Model - Graphic Editor



Step 4). Identify Operations

Extract verbs connected with an object.

Develop a graphic editor that can **draw** different geometric shapes such as line, circle and triangle. User can **select**, **move** or **rotate** a shape. To do so, editor **provides** user with a menu listing different commands. Individual shapes can be **grouped** together and can **behave** as a single shape.

Eliminate irrelevant operations.

- Develop - out of system boundary
- Provides - have broad semantics
- Behave – broad semantics

Following are selected operations:

- **Shape**

- Draw
- Select
- Move
- Rotate

- **Line**

- Draw
- Select
- Move
- Rotate

- **Circle**

- Draw
- Select
- Move
- Rotate

- **Triangle**

- Draw
- Select
- Move
- Rotate

- **Menu**

- Open
- Select
- Move
- Rotate

- **Group**

- Draw
- Select

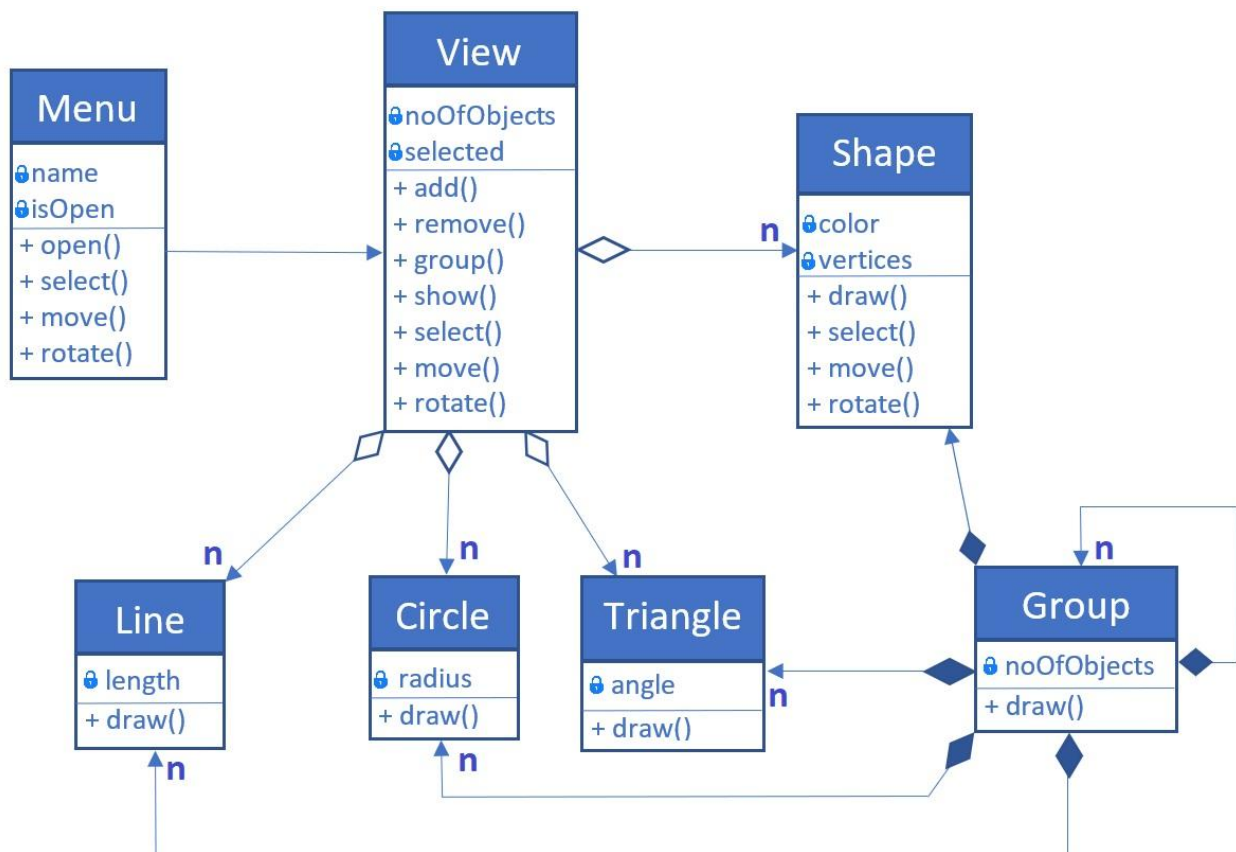
- Move
- Rotate

Extract operations using domain knowledge

- **View**

- Add
- Remove
- Group
- Show

Basic Object Model - Graphic Editor



Step 5). Identify Inheritance

Search "is a kind of" by looking at keywords like "such as", "for example", etc

"...shapes such as line, circle and triangle..."

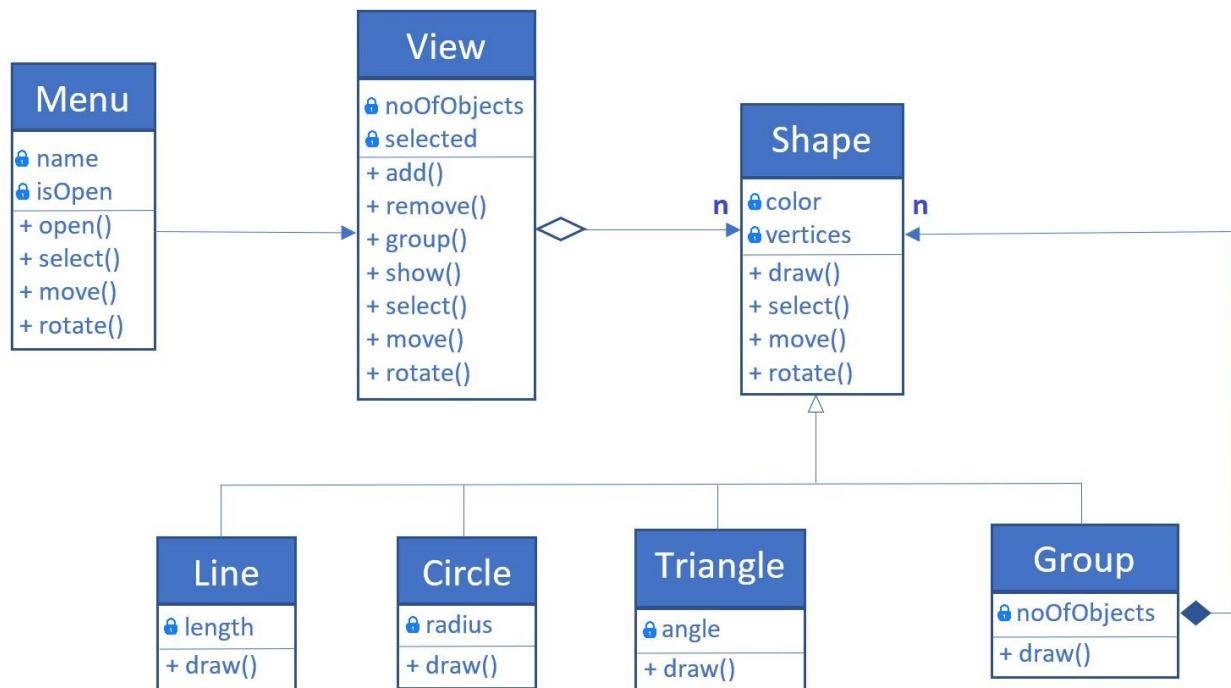
- Line, Circle and Triangle inherits from Shape

By analyzing requirements

"Individual shapes can be grouped together and can behave as a single shape"

- Group inherits from Shape

Refined Object Model - Graphic Editor



Rõ ràng cách thức xây dựng mô hình miền Y CHANG như cách chúng ta đang tiếp cận: đó là tìm kiếm danh từ/cụm danh từ, tinh chỉnh (thêm/xoá) và vẽ sơ đồ chỉ có tên. Như vậy cách chúng ta đi theo tài liệu ICONIX là đang đúng hướng.

Kết luận

Mô hình hóa miền giữ vai trò then chốt trong việc biến những mô tả yêu cầu mơ hồ thành một "bản đồ khái niệm" rõ ràng và có thể giao tiếp được giữa tất cả các bên liên quan. Đây không phải là bước phụ, mà là nền móng tĩnh (static backbone) cho toàn bộ quy trình thiết kế phần mềm. Thông qua việc trích xuất danh từ từ yêu cầu, loại bỏ trùng lặp, chuẩn hóa thuật ngữ và xây dựng các quan hệ has-a, is-a, knows-about, domain model giúp nhóm thống nhất ngôn ngữ, tránh hiểu lầm và phát hiện sớm các lỗi hổng khái niệm. Case study Internet Bookstore cho thấy quy trình xây dựng domain model là lặp và tinh chỉnh liên tục: bắt đầu từ sơ đồ sơ khai, brainstorm để bổ sung/loại bỏ lớp, rồi áp dụng tổng quát hóa để đơn giản hóa cấu trúc. Kết quả là một mô hình vừa đủ, nhất quán và dễ mở rộng. Một domain model tốt sẽ làm cho các bước sau — use case, robustness analysis, sequence diagram và coding — trở nên trôi chảy và nhất quán. Ngược lại, một domain model yếu sẽ khiến toàn bộ hệ thống thiết kế trở nên rối rắm, tốn kém và dễ sụp đổ như một tòa nhà xây trên nền móng yếu.

Câu hỏi ôn tập

1. Domain model là gì và nó đóng vai trò gì trong quá trình phát triển phần mềm theo ICONIX?
2. Trình bày sự khác biệt giữa Domain Model và Database Model. Vì sao không nên nhầm lẫn hai mô hình này?
3. Vì sao Domain Modeling cần được thực hiện trước hoặc song song với Use Case?
4. Liệt kê và mô tả ngắn gọn ba loại quan hệ thường gặp trong domain model.
5. Vì sao không nên đưa các thành phần giao diện hoặc kỹ thuật (như LoginForm, API, nút bấm...) vào domain model?
6. Cho ví dụ: “ShoppingCart có LineItem”, “BookCatalog có Book”, “Order có Checkout”. Xác định câu nào là quan hệ đúng, câu nào sai và giải thích ngắn gọn.
7. Trong case study Internet Bookstore, tại sao lớp Author bị loại bỏ khỏi domain model, còn OrderHistory lại được thêm vào?
8. Giải thích tại sao Checkout là một “động từ giả danh danh từ” và hệ quả nếu giữ nó trong domain model.
9. Nêu quy trình 7 bước để xây dựng domain model thực tế theo ICONIX.
10. Hãy chọn một hệ thống đơn giản (ví dụ: quản lý thư viện hoặc cửa hàng thực phẩm online) và mô tả 3 lớp miền chính cùng mối quan hệ giữa chúng.